

Snog's Enemy AI System

Version: 1.0.0

Unity: Unity 6 (6000.x) / 2023 LTS compatible

NavMesh: Built-in NavMesh (Navigation window)



Table of Contents

- [1. What's Included](#)
- [2. System Requirements](#)
- [3. Installation](#)
- [4. 5-Minute Quick Start](#)
- [5. Setup Wizard \(Recommended\)](#)
- [6. Components Overview](#)
 - [6.1 VisionSensor](#)
 - [6.2 HearingSensor](#)
 - [6.3 ThreatMemory](#)
 - [6.4 EnemyAIController](#)
 - [6.5 SuspicionSystem](#)
- [7. Inspectors & Gizmos \(Editor UX\)](#)
- [8. Player Setup](#)
 - [8.1 PerceptionTarget](#)
 - [8.2 Footsteps / SoundEmitter](#)
- [9. Patrol Modes](#)
- [10. Hide-From-Player \(Bracken-like\)](#)
- [11. Suspicion & Search Behavior](#)
- [12. Performance Tips](#)
- [13. Lightweight API Reference](#)
- [14. Troubleshooting & FAQ](#)
- [15. Suggested Workflows & Samples](#)
- [16. Support](#)





1. What's Included

Runtime

- **VisionSensor** — Focus/Peripheral FOV zones, robust LOS (**MultiRay** / Capsule), detection with curves.
- **HearingSensor** — Distance/occlusion aware, **prioritization** & per-type **cooldowns**.
- **ThreatMemory** — Last known position + **velocity** with simple **prediction**.
- **EnemyAIController** — **NavMesh**-driven AI with **Patrol (Waypoints/Random)**, **Investigate**, **Chase**, **Search**, **Hide-from-Player** (Camera.main).
- **PerceptionTarget** — Marks player (or any detectable entity).
- **SoundSystem** + **SoundEmitter** — Event bus and source component for hearing.
- **SuspicionSystem** — Lightweight world “suspicion discs” with decay.
- **FootstepTestEmitter** (optional) — Press Space to emit test sounds.

Editor

- **Setup Wizard** — One-click layers/tags, player/enemy setup, SuspicionSystem dropper, NavMesh helpers.
- Custom Inspectors for **EnemyAIController**, **VisionSensor**, **HearingSensor** with collapsible sections, contextual fields, debug toggles.
- Gizmos for FOV cones, hearing radii, patrol regions, waypoints, last random point.



2. System Requirements

- **Unity:** Unity 6 (6000.x) or 2023 LTS
- **Physics:** 3D physics enabled
- **NavMesh:** Built-in Navigation window (Window ▶ AI ▶ Navigation)
- **Layers:**
 - **Ground** — walkable surfaces
 - **Obstacles** — vision/hearing occluders
 - **PerceptionTarget** — player/targets detectable by Vision

The **Setup Wizard** can create these for you.



3. Installation

1. Import the package into your Unity project.
2. Open any **3D** scene (URP/HDRP/Built-in supported).



4. 5-Minute Quick Start

1. Open Setup Wizard

SnogTools ► AI ► Stealth Setup Wizard

2. Create Layers & Tag

Click **Create / Ensure Layers & Tag** to create `Ground` , `Obstacles` , `PerceptionTarget` , and `Player` tag.

3. Mark Player

Assign your Player GameObject and click **Mark Player & Add Components**
(Adds `PerceptionTarget` , `SoundEmitter` , optional `FootstepTestEmitter` , tags as `Player` , optionally sets `PerceptionTarget` layer.)

4. Create SuspicionSystem

Click **Create SuspicionSystem (Scene)**.

5. Configure Enemies

Drag enemy prefab(s)/instance(s) into the Wizard and click **Configure Selected Enemies**
(Adds/sets `NavMeshAgent` , `EnemyAIController` , `VisionSensor` , `HearingSensor` , `ThreatMemory` , plus masks/layers.)

6. Bake NavMesh

Click **Open Navigation Window** → Mark floors/terrain as **Ground** → **Bake**.

7. Press Play

- Look at enemies with **Camera.main**: with Hide enabled, they **freeze** or **flee to occlusion**.
- Press **Space** (with `FootstepTestEmitter`) to emit a test footstep; enemies **investigate** and **search**.



5. Setup Wizard (Recommended)

Menu: SnogTools ► AI ► Stealth Setup Wizard

Project Setup

- **Create / Ensure Layers & Tag**
 - **Ground:** walkable
 - **Obstacles:** occluders for vision/hearing/player-watch logic
 - **PerceptionTarget:** player/targets layer
 - **Player Tag:** convenience tag for your player object

Scene Setup

- **Player Object**
 - `PerceptionTarget` — required for Vision
 - `SoundEmitter` — needed for Hearing
 - `FootstepTestEmitter` — press **Space** in Play Mode to emit a Footstep sound
- **SuspicionSystem**

Create one in the scene; manages suspicion nodes with decay.

Enemy Setup

- **Ensure Components:** `NavMeshAgent` , `ThreatMemory` , `VisionSensor` , `HearingSensor` , `EnemyAIController`
- **Configure Masks:**
 - `Vision targetMask = PerceptionTarget`
 - `Vision/Hearing occluderMask = Obstacles`
 - `Controller groundLayer = Ground`
 - `Controller playerOccluderMask = Obstacles`
- **Random Patrol:** Optionally **Avoid Player LoS** (stealthier wandering)

Navigation

- **Open Navigation Window** to bake.
- **Select All NavMeshAgents** to tune settings if needed.



6. Components Overview

6.1 VisionSensor

- **Two FOV zones**
 - **Focus Cone** (narrow, faster detection)
 - **Peripheral Cone** (wide, slower detection)
- **LOS Modes:** `SingleRay` , `MultiRay` (default), `Capsule`
- **Curves:** Angle & Distance weighting per zone
- **Gizmos:** Focus + Peripheral cones, forward ray
- **Events:** `OnTargetSpotted` , `OnTargetLost`

Key Fields

- `focusFOV` , `peripheralFOV` , `viewDistance` , `eyeHeight`
- `focusTimeToFullDetect` , `peripheralTimeToFullDetect`
- `focusAngleWeight` , `focusDistanceWeight`
- `peripheralAngleWeight` , `peripheralDistanceWeight`
- `detectionDecayPerSecond` , `scanInterval`
- `losMode` , `losSamples` , `losSampleRadius` , `losCapsuleRadius`
- `targetMask` , `occluderMask`



Tip

Keep `peripheralFOV` $\geq$ `focusFOV` .



6.2 HearingSensor

- Radius scales by **loudness**; optional **occlusion** attenuation.
- **Prioritization** & per-type **cooldowns** to avoid event spam.
- **Event:** `OnHeardSound(SoundEvent evt)`

Key Fields

- `baseHearingRadius` , `minLoudness` , `occluderMask` , `occlusionAttenuation`

- `maxEventsPerFrame`
- **Weights:** `footstepWeight`, `gunshotWeight`, `impactWeight`, `voiceWeight`, `customWeight`
- **Cooldowns:** `footstepCooldown`, `gunshotCooldown`, `impactCooldown`, `voiceCooldown`, `customCooldown`



6.3 ThreatMemory

- Stores **LastKnownPosition** & **LastKnownVelocity**
- **Prediction:** `GetPredictedPosition()` uses `predictionLeadTime`
- **Decay:** forgets after `forgetAfterSeconds`



6.4 EnemyAIController

- **States:** `Idle`, `Patrol`, `Investigate`, `Chase`, `Search`
- **Patrol**
 - **Waypoints** — cycles array with wait time
 - **Random** — samples valid **NavMesh** points projected to **Ground** (down-ray), optional **Avoid Player LoS**
- **Investigate/Search**
 - Investigate moves to **heard** position; raises **Suspicion**
 - Search around **last known** or **strongest Suspicion** node
- **Chase**
 - Repaths every interval; **predicts** target movement with ThreatMemory
- **Hide From Player (Bracken-like)**
 - If **Camera.main** sees the enemy in FOV/distance **with no occluder** → either **Freeze** or **FleeToOcclusion**

Key Fields

- Patrol: `patrolMode`, `patrolPoints`, `randomPatrolRadius`, `groundLayer`, `avoidPlayerLoSInRandomPatrol`
- Investigate/Search: `investigateDuration`, `searchRadius`, `searchDuration`

- Chase: `repathInterval` , `giveUpAfterSeconds`
- Prediction/Suspicion: `predictionScaleDistance` , `maxPredictionLeadScale` , `suspicionQueryDistance` , `suspicionThreshold`
- Hide: `hideFromPlayer` , `hideBehavior` , `playerFOV` , `watchedDistance` , `hideRepathCooldown` , `hideSampleAttempts` , `hideSearchRadius` , `playerOccluderMask`



6.5 SuspicionSystem

- Manages world-space **suspicion nodes** with **radius**, **value**, and **decay**.
- Raised by **Hearing** and **Vision** drop-off (config in controller).
- AI queries strongest nearby nodes during **Search** (and can bias Patrol if desired).



7. Inspectors & Gizmos (Editor UX)

EnemyAIController (Custom Inspector)

- Collapsible sections:
 - References
 - Patrol (Waypoints vs Random shows contextual fields)
 - Investigate/Search
 - Chase
 - **Prediction/Suspicion**
 - **Hide From Player**
 - Gizmos / Debug

VisionSensor (Custom Inspector)

- Targets & FOV; Detection curves; Budget; **Gizmo toggles**
- **Runtime**: Detection % progress bar, **LOS Test** against a chosen target

HearingSensor (Custom Inspector)

- Hearing; Budget; Gizmos
- **Runtime**: **Emit Test Sound** (at sensor or nearby), **Last Heard** (type, loudness, distance, age)

Gizmos

- FOV cones (Focus + Peripheral), forward ray
- Hearing radius disc
- Patrol region disc & last random point, waypoint nodes and path



8. Player Setup

8.1 PerceptionTarget

Attach to any object that should be **seen** by enemies (e.g., Player).

Optional: assign a `targetPoint` (e.g., head).

8.2 Footsteps / SoundEmitter

Emit sounds for footsteps, impacts, doors, etc.

```
// PerceptionTarget is a simple marker component.
// No configuration is required; wizard can add it automatically.

using SnogTools.AI;
using UnityEngine;

public class SimpleFootsteps : MonoBehaviour
{
    public SoundEmitter emitter;
    private void Reset()
    {
        emitter = GetComponent<SoundEmitter>();
    }

    public void OnFootstep()
    {
        // Loudness ~1.0 (walk), ~2.0 (run); tune maxRange for your level
        emitter.Emit(loudness: 1.2f, maxRange: 12f, type: SoundType.Footstep);
    }
}
```

Quick test: The included `FootstepTestEmitter` lets you press **Space** in Play Mode to emit a Footstep.



9. Patrol Modes

Waypoints

- Set `patrolPoints` in order; the agent cycles through, waiting `waitAtDestination` at each.

Random

- Samples valid NavMesh points within `randomPatrolRadius`, projects onto **Ground** via downward ray.
- **Avoid Player LoS** (optional): prefers points **occluded** from **Camera.main** (stealthier).

Common Settings

- `waypointTolerance` — how close is considered “arrived”
- `randomWaitMin/Max` — natural idle between moves
- `randomMaxAttempts` — robustness in cluttered spaces



10. Hide-From-Player

If **Camera.main** looks at the enemy within `playerFOV` and `watchedDistance`, **and there’s no occluder**, the enemy reacts instantly—even before it detects the player.

Behaviors

- **Freeze:** stop pathing (great for horror “statue” behavior)
- **FleeToOcclusion:** sample nearby NavMesh points that are **not visible** from the camera and move there (cooldown-based)

Setup

- On **EnemyAIController**:
 - Enable **Hide From Player**
 - Choose **Freeze** or **FleeToOcclusion**

- Set **playerOccluderMask** to your Obstacle layers



11. Suspicion & Search Behavior

- **Hearing** events:
 - Raise **Suspicion** at the sound's position
 - Switch to **Investigate** (move directly there)
- **Vision** drop-off:
 - If detection was significant and target is lost, raise **smaller Suspicion** at last known position
- **Search**:
 - If memory exists → search around **last known**
 - Else → search around **strongest Suspicion** nearby
 - Times out → returns to **Patrol**



12. Performance Tips

Vision

- Increase `scanInterval` (e.g., 0.15–0.25s) for many agents
- **MultiRay** with 2–3 samples is robust and efficient
- Reduce `viewDistance` to match your level scale

Hearing

- Limit `maxEventsPerFrame` (e.g., 2–4)
- Use per-type **cooldowns** to reduce spam

NavMesh

- Keep `repathInterval` ~0.2–0.4s
- Ensure agent radius/height match environment

Occluder Masks

- Keep **Obstacles** layers tight and correct to minimize ray hits



13. Lightweight API Reference

VisionSensor

```
public class VisionSensor : MonoBehaviour
{
    public LayerMask targetMask;
    public LayerMask occluderMask;

    public float focusFOV;
    public float peripheralFOV;
    public float viewDistance;
    public float eyeHeight;

    public float focusTimeToFullDetect;
    public AnimationCurve focusAngleWeight;
    public AnimationCurve focusDistanceWeight;

    public float peripheralTimeToFullDetect;
    public AnimationCurve peripheralAngleWeight;
    public AnimationCurve peripheralDistanceWeight;

    public float detectionDecayPerSecond;
    public float scanInterval;

    public LosMode losMode; // SingleRay | MultiRay | Capsule
    public int losSamples;
    public float losSampleRadius;
    public float losCapsuleRadius;

    public event Action<Transform> OnTargetSpotted;
    public event Action OnTargetLost;

    public Transform CurrentTarget { get; private set; }
    public float CurrentDetection { get; private set; }

    public bool HasLineOfSight(Transform target);
    public bool HasLineOfSight(Vector3 worldPoint);
```

```
}
```

HearingSensor

```
public class HearingSensor : MonoBehaviour
{
    public float baseHearingRadius;
    public float minLoudness;
    public LayerMask occluderMask;
    public float occlusionAttenuation;

    public int maxEventsPerFrame;
    public float footstepWeight;
    public float gunshotWeight;
    public float impactWeight;
    public float voiceWeight;
    public float customWeight;

    public float footstepCooldown;
    public float gunshotCooldown;
    public float impactCooldown;
    public float voiceCooldown;
    public float customCooldown;

    public event Action<SoundEvent> OnHeardSound;
}
```

ThreatMemory

```
public class ThreatMemory : MonoBehaviour
{
    public float forgetAfterSeconds;
    public float predictionLeadTime;

    public Vector3 LastKnownPosition { get; private set; }
    public Vector3 LastKnownVelocity { get; private set; }
    public bool HasLastKnownPosition { get; private set; }

    public void SetLastKnown(Vector3 pos, Vector3 velocity);
    public void SetLastKnownPosition(Vector3 pos);
}
```

```

    public void ClearPosition();
    public void Touch();
    public void Tick();

    public Vector3 GetPredictedPosition(float leadTimeScale = 1f);
}

```

EnemyAIController

```

public class EnemyAIController : MonoBehaviour
{
    public event Action<AIState, AIState> OnStateChanged;

    public void ForceInvestigate(Vector3 position);
    public void ForceChase(Transform target);
    public void Abort();
}

```

Sound

```

public enum SoundType
{
    Footstep,
    Gunshot,
    Impact,
    Voice,
    Custom
}

public struct SoundEvent
{
    public Vector3 worldPosition;
    public float loudness;
    public float maxRange;
    public SoundType type;
    public UnityEngine.Object source;
}

public static class SoundSystem
{

```

```

    public static event Action<SoundEvent> OnSound;
    public static void Raise(SoundEvent evt);
}

public class SoundEmitter : MonoBehaviour
{
    public void Emit(
        float loudness,
        float maxRange,
        SoundType type = SoundType.Custom
    );
}

```



14. Troubleshooting & FAQ

Enemy isn't moving

- Ensure a **NavMeshAgent** is on the enemy and **NavMesh is baked** (Window ▸ AI ▸ Navigation ▸ Bake).
- Confirm the agent is **on the NavMesh** (toggle NavMesh visualization).
- If Hide mode is **Freeze** and the camera looks at the enemy, it will stop—try **FleeToOcclusion** or look away.

Enemy never sees the player

- Player must have **PerceptionTarget** and be on the **PerceptionTarget** layer.
- Vision `targetMask` must include **PerceptionTarget**.
- Ensure **Obstacles** aren't accidentally included in the target mask.
- Adjust `peripheralFOV` / `viewDistance` for your scale.

Hearing feels inconsistent

- Align `baseHearingRadius` and event `maxRange` with level scale.
- Tune `occlusionAttenuation` (lower = more muffled; higher = less).
- Check `maxEventsPerFrame` and per-type cooldowns.

Hide-from-Player doesn't trigger

- Must use **Camera.main** (or assign `playerEye` override).
- Tune `playerFOV` , `watchedDistance` , and `playerOccluderMask` .
- Ensure no occluder is blocking the camera-to-enemy line when testing.

Random patrol picks odd points

- Mark floors as **Ground**; assign Controller's `groundLayer` .
- Increase `randomMaxAttempts` .
- Temporarily disable **Avoid Player LoS** to validate sampling.

Performance dips with many enemies

- Increase Vision `scanInterval` ; use **MultiRay** with 2–3 samples.
- Lower Hearing `maxEventsPerFrame` .
- Shorten Vision `viewDistance` .



15. Suggested Workflows & Samples

Basic Guard

- Peripheral FOV: 110°, Focus FOV: 50°, ViewDistance: 18–25
- Time to full: Focus 0.6s, Peripheral 1.2s
- Patrol: Waypoints + wait 1–2s

Sneaky Stalker (Bracken-like)

- Enable **Hide From Player** → **FleeToOcclusion**
- Random Patrol with **Avoid Player LoS**
- Slower peripheral detection, faster focus detection
- Smaller `searchRadius` to “linger around corners”

Noisy Arena

- Hearing max events/frame 2–3
- Footstep cooldown 0.25s, Gunshot 0.05s
- Impact weight 1.2, Voice 1.1

Turn on **Gizmo/Debug** toggles in inspectors to visualize FOV, hearing, patrol areas, and last random points for fast tuning.



16. Support

Support & Feature Requests

[Click here to send an email to me](#)